

Optimizer-Report

June 12, 2026

1 Optimizer

Optimizer Definition: An **optimizer** is a function that **adjusts the parameters** (weights and biases) of a model in a way that **reduces the loss function**. This is usually done through **gradient descent**, which calculates the negative of each parameter's partial derivatives to move in the direction of steepest descent.

Problem: Standard gradient descent is often insufficient because the loss function can become stuck in **local minima** or **saddle points** (points where the gradient is approaching 0 but is neither a minimum nor maximum). Therefore, different types of optimizers exist to combat this problem.

1.1 Stochastic Gradient Descent

Stochastic Gradient Descent (SGD) computes the negative gradient for a **subset of the data** (batch) instead of the entire dataset.

Advantages:

- Computing negative gradients by batches makes the loss function more likely to escape local minima or saddle points while still converging towards a global minimum
- Less computation power is required because the gradient is computed on a subset of the dataset, not the entire dataset

Disadvantages:

- SGD has a more unpredictable convergence behavior compared to gradient descent because it calculates the gradient for a subset of the dataset. It is, therefore, more prone to having a noisy and erratic convergence trajectory
- Its performance is also heavily dependent on the learning rate, where poor learning rates result in slower convergence towards a minima

1.2 Root Mean Square Propagation

Root Mean Square Propagation (RMSprop) attempts to improve SGD by implementing an **adaptive learning rate** for each parameter during training

Advantages:

- Maintains all the advantages of SGD while implementing adaptive learning rates, resulting in faster convergence and less erratic convergence behavior

Disadvantages:

- Although RMSprop improves from standard SGD, there are other optimizers such as Adam which improves on RMSprop by implementing a new concept called **momentum**. Without momentum, RMSprop still faces the same disadvantages as SGD to a certain extent

1.3 Adaptive Moment Estimation

Adaptive Moment Estimation (Adam) implements the **adaptive learning rate** from RMSprop while implementing **momentum** and **normalization**. Momentum uses the accumulation of past gradients to update the current gradient of the loss function, allowing the gradient to maintain movement in the same direction.

Advantages:

- Maintains all the advantages that RMSprop had with its adaptive learning rate and batching behavior
- Adam's implementation of momentum results in faster convergence, less erratic convergence behavior, and less oscillations of the loss function compared to previous optimizers.
- By normalizing the gradient, the loss function converges to the minima at a near constant rate regardless of the gradient size. This results in a smoother and less oscillatory convergence behavior

Disadvantages:

- For some scenarios, SGD can perform better than Adam
- There are many variants of Adam that perform better than the original Adam in specific scenarios

1.4 AdamW

AdamW uses the same concept as Adam but implements **weight decay** while updating the parameter instead of implementing it into the loss function.

Advantages:

- Implementing weight decay into the loss function can affect Adam's adaptive learning rate so by implementing it into the parameter updates, it ensures that the adaptive learning rate is not affected while adding regularization to prevent overfitting.
- It maintains all of Adam's advantages with momentum, adaptive learning rates, and batching

Disadvantages:

- AdamW takes significantly more computing power than simpler optimizers such as SGD
- The optimizer has a high sensitivity to initial hyperparameters such as weight decay and by choosing the nonoptimal hyperparameters, model performance can be negatively affected